نام دانشجو: ابوالفضل جلیل دوست

استاد: دکتر حمیدرضا لطفعلی شمیرانی

مستندات فنی پروژه

ابزارها، معماری و روند ساخت سامانه مدیریت وظایف و اعلان‌ها (Task Management API)

فناوری‌ها: NestJS · TypeORM · SQLite · JWT

۱. هدف این سند

سند «سناریوی پروژه» به این پرسش پاسخ می‌دهد که سامانه **چه کاری** انجام می‌دهد. این سند مکمل آن است و به این پرسش پاسخ می‌دهد که سامانه **چگونه** ساخته شده است: از چه ابزارها و کتابخانه‌هایی استفاده شده، ساختار کد چگونه سازمان‌دهی شده، پروژه در چه مراحلی شکل گرفته، و کیفیت آن با چه ابزارهایی (تست خودکار، Lint کد و CI) تضمین می‌شود.

- معرفی ابزارها و فناوری‌ها، همراه با نسخه و دلیل انتخاب هر کدام.
- شرح معماری ماژولار و ساختار پوشه‌های پروژه.
- شرح گام‌به‌گام روند ساخت پروژه از ایجاد ساختار اولیه تا استقرار CI Pipeline.
- نحوه‌ی پیکربندی، اجرا، تست و نگه‌داری پروژه.

۲. ابزارها و فناوری‌های مورد استفاده

۲.۱ محیط اجرا و زبان برنامه‌نویسی

ابزار	نسخه	کاربرد و دلیل انتخاب
Node.js	24.x	محیط اجرای جاوااسکریپت سمت سرور. نسخه‌ی ۲۴ به این دلیل الزامی شده که مجموعه‌ی تست‌ها روی نسخه‌های قدیمی‌تر به‌دلیل نحوه‌ی بارگذاری ماژول‌های ESM اجرا نمی‌شود.
TypeScript	6.x	زبان اصلی پروژه. سیستم نوع ایستا خطاها را پیش از اجرا آشکار می‌کند و دکوراتورهای مورد نیاز NestJS و TypeORM را پشتیبانی می‌کند.
npm	—	Package Manager؛ وابستگی‌ها در `package.json` تعریف و نسخه‌ی دقیق آن‌ها در `package-lock.json` قفل شده است.

۲.۲ فریم‌ورک و کتابخانه‌های اصلی (وابستگی‌های زمان اجرا)

ابزار	نسخه	کاربرد در پروژه
NestJS (@nestjs/core, common, platform-express)	12.x	فریم‌ورک اصلی سمت سرور؛ ساختار ماژولار، Dependency Injection، کنترلرها و سرویس‌ها را فراهم می‌کند. لایه‌ی HTTP بر پایه‌ی Express است.
TypeORM + @nestjs/typeorm	1.x	ORM؛ تعریف Entityها با دکوراتور، Repositoryها برای دسترسی به داده، و Migration برای مدیریت تغییرات Schema دیتابیس.

better-sqlite3	12.x	درایور دیتابیس SQLite؛ دیتابیس یک فایل محلی است و نیازی به نصب سرور جداگانه ندارد.
@nestjs/config	12.x	خواندن تنظیمات از فایل `.env` و اعتبارسنجی آن‌ها در زمان راه‌اندازی برنامه.
@nestjs/jwt + @nestjs/passport + passport-jwt	12.x / 0.7 / 4.x	صدور و اعتبارسنجی توکن‌های JWT و پیاده‌سازی Strategy احراز هویت (JwtStrategy) و Guard روت‌ها (JwtAuthGuard).
bcrypt	6.x	هش گذرواژه‌ها با Salt اختصاصی برای هر کاربر.
class-validator + class-transformer	0.15 / 0.5	اعتبارسنجی داده‌های ورودی از طریق دکوراتورها در DTOها، و تبدیل/فیلتر داده‌های خروجی پیش از ارسال به کاربر.
@nestjs/schedule	12.x	اجرای زمان‌بندی‌شده‌ی بررسی سررسید وظایف با دکوراتور `@Cron()` (هر پنج دقیقه یک‌بار).
@nestjs/swagger	12.x	تولید خودکار مستندات تعاملی OpenAPI در آدرس `/api/docs`.
rxjs, reflect-metadata, dotenv	—	وابستگی‌های زیرساختی NestJS؛ و بارگذاری `.env` برای CLI TypeORM.

۲.۳ ابزارهای توسعه، تست و کیفیت کد

ابزار	نسخه	کاربرد در پروژه
Nest CLI (@nestjs/cli, schematics)	12.x	ایجاد ساختار اولیه پروژه و ماژول‌ها، کامپایل (`nest build`) و اجرای حالت Development با Hot Reload.
Jest + ts-jest	30.x / 29.x	فریم‌ورک تست؛ دو پروژه‌ی جداگانه برای Unit Test و E2E Test و گزارش Coverage.
@nestjs/testing + supertest	12.x / 7.x	ساخت نمونه‌ی واقعی برنامه در تست‌ها و ارسال درخواست‌های HTTP واقعی به آن.
ESLint + typescript-eslint	9.x / 8.x	Lint کد با قواعد type-checked.
Prettier	3.x	فرمت یکنواخت کد (Single Quote و Trailing Comma).
ts-node	10.x	اجرای CLI TypeORM برای تولید، اجرا و بازگردانی Migrationها.
cross-env, rimraf	—	تنظیم متغیرهای محیطی به‌صورت مستقل از سیستم‌عامل، و پاک‌سازی پوشه‌ی `dist` پیش از کامپایل.
Git + GitHub Actions	—	کنترل نسخه، و CI Pipeline که در هر push اجرا می‌شود.

Swagger UI / curl	—	تست دستی Endpointهای API در طول توسعه.

Node.js 24 TypeScript NestJS 12 TypeORM SQLite (better-sqlite3) JWT Passport.js bcrypt class-validator nestjs/schedule Swagger / OpenAPI Jest Supertest ESLint Prettier GitHub Actions

۳. معماری و ساختار پروژه

پروژه بر پایه‌ی معماری ماژولار NestJS ساخته شده است. هر قابلیت (احراز هویت، کاربران، وظایف، دسته‌بندی‌ها و اعلان‌ها) یک ماژول مستقل است که Entity، سرویس، کنترلر و DTOهای خودش را در اختیار دارد.

مسیر	محتوا
`src/auth/`	Strategy JWT، Guard روت‌ها، ثبت‌نام و ورود
`src/users/`	Entity کاربر و روت‌های پروفایل
`src/todos/`	Entity وظیفه، جست‌وجو، فیلتر و صفحه‌بندی
`src/categories/`	Entity دسته‌بندی
`src/notifications/`	Entity اعلان و بررسی زمان‌بندی‌شده‌ی سررسیدها
`src/config/`	تنظیمات دیتابیس و اعتبارسنجی متغیرهای محیطی
`src/migrations/`	Migrationهای Schema دیتابیس
`src/decorators/`	دکوراتور `@CurrentUser()`
`src/interceptors/`	Interceptor `@Serialize(Dto)`
`src/app.setup.ts`	Prefix روت و اعتبارسنجی سراسری (مشترک با تست‌ها)
`src/swagger.ts`	پیکربندی مستندات OpenAPI
`src/main.ts`	نقطه‌ی شروع برنامه
`test/`	تست‌های E2E
`docs/`	مستندات پروژه
`.github/workflows/`	Workflow CI

۳.۱ لایه‌های برنامه

درخواست HTTP ◄ ValidationPipe ◄ JwtAuthGuard ◄ Controller ◄ Service ◄ Repository (TypeORM) ◄ SQLite

لایه	مسئولیت
Controller	فقط دریافت درخواست HTTP، استخراج پارامترها و فراخوانی سرویس؛ هیچ قاعده‌ی کسب‌وکاری در این لایه نیست.
Service	تمام قواعد کسب‌وکار (بررسی مالکیت، تکراری‌بودن، هش گذرواژه، تولید اعلان). این تفکیک باعث می‌شود سرویس‌ها به‌سادگی Unit Test شوند.
Repository	دسترسی به داده از طریق TypeORM؛ هر Query همواره به شناسه‌ی مالک محدود می‌شود.
DTO	تعریف شکل داده‌ی ورودی (با قواعد اعتبارسنجی) و داده‌ی خروجی (فقط فیلدهای مجاز).

۳.۲ اجزای مشترک کلیدی

Interceptor — @Serialize(Dto) خروجی

هر پاسخ پیش از ارسال از یک DTO عبور می‌کند و فقط فیلدهایی که با `@Expose()` علامت‌گذاری شده‌اند برگردانده می‌شوند. به همین دلیل، با اینکه Entity کاربر شامل هش گذرواژه است، این مقدار هرگز به کاربر نمی‌رسد.

@CurrentUser() — دکوراتور پارامتر

کاربری را که JwtStrategy از روی توکن شناسایی کرده به کنترلر تزریق می‌کند؛ در نتیجه هویت کاربر هرگز از URL یا Body درخواست خوانده نمی‌شود.

configureApp() — پیکربندی مشترک

Prefix `/api/v1` و ValidationPipe سراسری (با گزینه‌های `whitelist`، `forbidNonWhitelisted` و `transform`) در یک تابع تعریف شده‌اند تا هم برنامه‌ی اصلی و هم تست‌های E2E دقیقاً با یک پیکربندی اجرا شوند.

۴. روند ساخت پروژه

پروژه در گام‌های زیر و به‌صورت تدریجی ساخته شده است. هر گام روی گام قبلی بنا شده و با تست‌های خودکار پشتیبانی می‌شود.

۱. ساختار اولیه ◄ ۲. پیکربندی ◄ ۳. دیتابیس ◄ ۴. احراز هویت ◄ ۵. کاربران ◄ ۶. دسته‌بندی و وظایف ◄ ۷. اعلان‌ها ◄ ۸. مستندسازی ◄ ۹. تست ◄ ۱۰. CI

گام ۱ ایجاد ساختار اولیه‌ی پروژه

ساختار اولیه پروژه با Nest CLI (دستور `nest new`) ایجاد شد و ماژول‌ها، کنترلرها و سرویس‌ها با دستور `nest generate` اضافه شدند. تمام وابستگی‌ها بر پایه‌ی NestJS 12 هستند.

گام ۲ پیکربندی و متغیرهای محیطی

ماژول ConfigModule به‌صورت سراسری بارگذاری شد و تابع `validateEnv` نوشته شد تا در صورت نبودن یا کوتاه‌بودن Secret JWT، برنامه همان ابتدا با پیام خطای روشن متوقف شود (نه در اولین ورود کاربر). فایل `.env` در `.gitignore` قرار دارد و فقط `.env.example` در ریپازیتوری نگه داشته می‌شود.

گام ۳ دیتابیس، Entityها و Migrationها

Entityها با دکوراتورهای TypeORM تعریف شدند و گزینه‌ی `synchronize` عمداً غیرفعال شد؛ به‌جای آن هر تغییر Schema دیتابیس از طریق یک Migration قابل بازبینی و قابل بازگشت اعمال می‌شود. Migrationها در زمان راه‌اندازی به‌صورت خودکار اجرا می‌شوند (`migrationsRun: true`)، بنابراین فایل دیتابیس در اولین اجرا ساخته می‌شود.

گام ۴ احراز هویت

روت‌های ثبت‌نام و ورود پیاده‌سازی شدند. گذرواژه با bcrypt هش می‌شود و پس از ورود موفق یک توکن JWT صادر می‌گردد. JwtStrategy در هر درخواست کاربر را دوباره از دیتابیس می‌خواند تا توکن یک حساب حذف‌شده بلافاصله بی‌اعتبار شود.

گام ۵ کاربران و پروفایل

روت‌های `GET/PATCH /users/me` اضافه شدند. ایمیل در ذخیره و مقایسه به حروف کوچک نرمال می‌شود و تغییر گذرواژه بدون ارسال گذرواژه‌ی فعلی ممکن نیست (هم در DTO و هم در سرویس بررسی می‌شود).

گام ۶ دسته‌بندی‌ها و وظایف

ماژول دسته‌بندی‌ها (با یکتایی نام برای هر کاربر) و ماژول وظایف (با اولویت، تاریخ سررسید و دسته‌بندی) اضافه شدند. فهرست وظایف از فیلتر، جست‌وجو در عنوان، مرتب‌سازی و صفحه‌بندی پشتیبانی می‌کند. فیلد مرتب‌سازی به یک فهرست مجاز محدود و تعداد نتایج هر صفحه حداکثر ۱۰۰ است. تمام Queryها به شناسه‌ی مالک محدود شده‌اند.

گام ۷ سامانه‌ی اعلان‌ها و زمان‌بند

Entity اعلان و روت‌های مدیریت آن اضافه شدند. اعلان خوش‌آمدگویی هنگام ثبت‌نام و اعلان تکمیل هنگام انجام وظیفه ساخته می‌شود. با ScheduleModule و دکوراتور `@Cron()`، متد `scanDueDates` هر پنج دقیقه وظایف نزدیک به سررسید یا سررسیدگذشته را بررسی می‌کند. این متد زمان جاری را به‌عنوان پارامتر می‌پذیرد تا بدون انتظار واقعی تست شود. شکست در ساخت اعلان، عملیات اصلی (مثل ثبت‌نام) را متوقف نمی‌کند و فقط هشدار ثبت می‌شود.

گام ۸ مستندسازی API

با @nestjs/swagger و افزونه‌ی کامپایلر آن (`introspectComments`)، مستندات تعاملی از روی DTOها و توضیحات کد تولید می‌شود و در آدرس `/api/docs` با امکان وارد کردن توکن در دسترس است. فایل `README.md` نیز شامل راهنمای اجرا، مرجع کامل API و نمودار مدل داده است.

گام ۹ تست‌های خودکار

برای هر سرویس Unit Test و برای هر جریان اصلی تست E2E نوشته شد (جزئیات در بخش ۷).

گام ۱۰ CI

یک Workflow در GitHub Actions اضافه شد که در هر push و Pull Request، Lint، کامپایل و تست‌ها را اجرا می‌کند (جزئیات در بخش ۸).

۴.۱ Migrationهای دیتابیس

Migration	محتوا
InitialSchema	ایجاد جداول کاربران و وظایف.
RenameGmailToEmail	تعیین ستون `email` در جدول کاربران.
AddCategoriesAndTodoMetadata	ایجاد جدول دسته‌بندی‌ها و فیلدهای اولویت، تاریخ سررسید و دسته‌بندی برای وظایف.
AddNotifications	ایجاد جدول اعلان‌ها و روابط آن با کاربران و وظایف.

۵. پیکربندی و اجرای پروژه

۵.۱ متغیرهای محیطی

متغیر	الزامی	پیش‌فرض	توضیح
JWT_SECRET	بله	—	Secret امضای توکن‌ها؛ حداقل ۱۶ نویسه
JWT_EXPIRES_IN	خیر	1d	مدت اعتبار (Expire) توکن
DATABASE_PATH	خیر	data/university-project.sqlite	مسیر فایل دیتابیس؛ `:memory:` برای دیتابیس موقت
PORT	خیر	3000	پورت برنامه
DUE_SOON_WINDOW_HOURS	خیر	24	بازه‌ی زمانی «نزدیک‌شدن سررسید» بر حسب ساعت

۵.۲ مراحل اجرا

```
npm install
cp .env.example .env        # then set JWT_SECRET
npm run start:dev

# API:   http://localhost:3000/api/v1
# Docs:  http://localhost:3000/api/docs
```

۵.۳ دستورهای پرکاربرد

دستور	کاربرد
npm run start:dev	اجرا در حالت Development با Hot Reload
npm run build / npm run start:prod	کامپایل به پوشه‌ی `dist` و اجرای نسخه‌ی Production
npm test	اجرای تمام تست‌ها (Unit و E2E)
npm run test:cov	اجرای تست‌ها همراه با گزارش Coverage
npm run lint	Lint کد با ESLint
npm run typeorm:generate-migration --name=X	تولید Migration جدید از روی تغییرات Entityها
npm run typeorm:revert-migration	بازگردانی آخرین Migration

۶. Endpointهای API

Prefix تمام روت‌ها `/api/v1` است. به‌جز ثبت‌نام و ورود، همه‌ی روت‌ها به `Authorization: Bearer <token>` Header نیاز دارند.

ماژول	روت‌ها	توضیح
احراز هویت	POST /auth/signup · POST /auth/login	ثبت‌نام و ورود
کاربران	GET /users/me · PATCH /users/me	مشاهده و ویرایش پروفایل
دسته‌بندی‌ها	GET · POST /categories GET · PATCH · DELETE /categories/:id	مدیریت کامل دسته‌بندی‌ها
وظایف	GET · POST /todos GET · PATCH · DELETE /todos/:id	مدیریت وظایف با فیلتر، جست‌وجو و صفحه‌بندی
اعلان‌ها	GET /notifications · PATCH /notifications/read-all PATCH /notifications/:id/read · DELETE /notifications/:id	مشاهده، خوانده‌شدن و حذف اعلان‌ها

۶.۱ نمونه درخواست‌ها و پاسخ‌ها

در این بخش، مهم‌ترین Endpointها همراه با Body درخواست، پاسخ موفق و خطاهای احتمالی آورده شده‌اند. نمونه‌ها از اجرای واقعی برنامه گرفته شده‌اند. فهرست کامل همه‌ی Endpointها به‌صورت تعاملی در Swagger (آدرس `/api/docs`) در دسترس است.

همه‌ی خطاها شکل یکسانی دارند. در خطای اعتبارسنجی (400)، فیلد `message` آرایه‌ای از پیام‌های خطاست:

```
{
  "message": [
    "email must be an email",
    "password must be longer than or equal to 8 characters"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

POST `/api/v1/auth/signup`

ثبت‌نام کاربر جدید. پس از ثبت‌نام، توکن مستقیماً برگردانده می‌شود.

Body درخواست

```
{
  "username": "ali",
  "email": "ali@example.com",
  "password": "password123"
}
```

پاسخ موفق — 201 Created

```
{
  "user": { "id": 1, "username": "ali", "email": "ali@example.com" },
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

کد خطا	زمان رخ‌دادن
400	ایمیل نامعتبر، یا گذرواژه کوتاه‌تر از ۸ نویسه
409	ایمیل تکراری: `"An account with this email already exists"`

POST `/api/v1/auth/login`

ورود و دریافت توکن.

Body درخواست

```
{
  "email": "ali@example.com",
  "password": "password123"
}
```

پاسخ موفق — 200 OK

```
{
  "user": { "id": 1, "username": "ali", "email": "ali@example.com" },
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

کد خطا	زمان رخ‌دادن
400	ایمیل یا گذرواژه ارسال نشده یا فرمت ایمیل نادرست است
401	ایمیل یا گذرواژه نادرست: `"Invalid email or password"`

POST `/api/v1/todos` نیاز به توکن

ایجاد وظیفه‌ی جدید. فقط `title` الزامی است؛ مقدار پیش‌فرض `priority` برابر `medium` است.

Body درخواست

```
{
  "title": "write thesis",
  "priority": "high",
  "dueDate": "2026-11-10T09:00:00.000Z",
  "categoryId": 1
}
```

پاسخ موفق — 201 Created

```
{
  "id": 1,
  "title": "write thesis",
  "done": false,
  "priority": "high",
  "dueDate": "2026-11-10T09:00:00.000Z",
  "category": { "id": 1, "name": "University" }
}
```

کد خطا	زمان رخ‌دادن
400	عنوان خالی، اولویت نامعتبر یا ارسال فیلد ناشناخته
401	توکن ارسال نشده یا نامعتبر است
404	دسته‌بندی با این شناسه وجود ندارد یا متعلق به کاربر نیست

GET `/api/v1/todos?priority=high&page=1&limit=10` نیاز به توکن

فهرست وظایف همراه با فیلتر و صفحه‌بندی.

پاسخ موفق — 200 OK

```
{
  "data": [
    {
      "id": 1,
      "title": "write thesis",
      "done": false,
      "priority": "high",
      "dueDate": "2026-11-10T09:00:00.000Z",
      "category": { "id": 1, "name": "University" }
    }
  ],
  "total": 1,
  "page": 1,
  "limit": 10,
  "totalPages": 1
}
```

کد خطا	زمان رخ‌دادن
400	مقدار نامعتبر برای پارامترها (مثلاً `limit` بیشتر از ۱۰۰)
401	توکن ارسال نشده یا نامعتبر است

PATCH `/api/v1/todos/:id` نیاز به توکن

ویرایش وظیفه؛ فقط فیلدهای ارسال‌شده تغییر می‌کنند. با `done: true` اعلان تکمیل ساخته می‌شود.

Body درخواست

```
{
  "done": true
}
```

پاسخ موفق — 200 OK

```
{
  "id": 1,
  "title": "write thesis",
  "done": true,
  "priority": "high",
  "dueDate": "2026-11-10T09:00:00.000Z",
  "category": { "id": 1, "name": "University" }
}
```

کد خطا	زمان رخ‌دادن
400	مقدار نامعتبر یا فیلد ناشناخته
401	توکن ارسال نشده یا نامعتبر است
404	وظیفه وجود ندارد یا متعلق به کاربر نیست: `"No task found with id 999"`

GET `/api/v1/notifications?unreadOnly=true` نیاز به توکن

فهرست اعلان‌های خوانده‌نشده‌ی کاربر.

پاسخ موفق — 200 OK


```
{
  "data": [
    {
      "id": 1,
      "type": "welcome",
      "message": "Welcome, ali!",
      "readAt": null,
      "createdAt": "2026-10-31T11:58:40.000Z",
      "todo": null
    },
    {
      "id": 2,
      "type": "todo_completed",
      "message": "\"write thesis\" was completed",
      "readAt": null,
      "createdAt": "2026-10-31T11:58:40.000Z",
      "todo": { "id": 1 }
    }
  ],
  "total": 2,
  "page": 1,
  "limit": 10,
  "totalPages": 1
}
```


کد خطا	زمان رخدادن
400	مقدار نامعتبر برای پارامترها (مثلاً `type` ناشناخته)
401	توکن ارسال نشده یا نامعتبر است

PATCH `/api/v1/users/me` نیاز به توکن

ویرایش پروفایل. برای تغییر گذرواژه، ارسال `oldPassword` الزامی است.

Body درخواست

```
{
  "password": "newpassword1",
  "oldPassword": "password123"
}
```

پاسخ موفق — 200 OK

```
{
  "id": 1,
  "username": "ali",
  "email": "ali@example.com"
}
```

کد خطا	زمان رخ‌دادن
400	گذرواژه‌ی فعلی ارسال نشده: `"oldPassword is required when changing password"`
401	گذرواژه‌ی فعلی نادرست است
409	ایمیل جدید متعلق به حساب دیگری است

Endpointهای حذف (`DELETE`) در صورت موفقیت کد 204 No Content و بدون Body برمی‌گردانند. `PATCH /notifications/read-all` تعداد اعلان‌های تغییرکرده را برمی‌گرداند: `{ "updated": 2 }`.

۷. تست و تضمین کیفیت

مجموعه‌ی تست‌ها با Jest نوشته شده و در دو پروژه‌ی جداگانه سازمان‌دهی شده است. در آخرین اجرا، تمام **۹۱ تست** در **۸ مجموعه** با موفقیت گذرانده شدند و Coverage کد حدود **٪۹۵** است.

نوع تست	تعداد	روش
Unit Test	۴۷	Repositoryها Mock می‌شوند و منطق هر سرویس به‌تنهایی بررسی می‌شود: هش گذرواژه، تشخیص تکرار، محدودسازی به مالک، ساخت Query و بررسی سررسید با یک زمان ثابت.
E2E Test	۴۴	برنامه‌ی واقعی روی یک دیتابیس SQLite In-Memory و تازه برای هر فایل اجرا می‌شود و درخواست‌های HTTP واقعی با supertest ارسال می‌شوند. از آنجا که Schema دیتابیس با اجرای Migrationها ساخته می‌شود، هر اجرای تست صحت Migrationها را نیز تأیید می‌کند.

فایل تست	تعداد	محدوده
auth.service.spec.ts	۵	ثبت‌نام، ورود، اعلان خوش‌آمدگویی
users.service.spec.ts	۸	به‌روزرسانی پروفایل و تغییر گذرواژه
categories.service.spec.ts	۴	یکتایی نام و محدودسازی به مالک
todos.service.spec.ts	۱۶	فیلتر، مرتب‌سازی، صفحه‌بندی و اعلان تکمیل
notifications.service.spec.ts	۱۴	بررسی سررسید، جلوگیری از تکرار، خوانده‌شدن
auth.e2e-spec.ts	۱۲	جریان کامل احراز هویت و پروفایل
todos.e2e-spec.ts	۲۰	جریان کامل وظایف و دسته‌بندی‌ها و جداسازی کاربران
notifications.e2e-spec.ts	۱۲	جریان کامل اعلان‌ها و اجرای مستقیم بررسی سررسید

علاوه بر تست‌ها، ESLint با قواعد type-checked و Prettier یکنواختی و کیفیت کد را تضمین می‌کنند. همچنین برای قواعد امنیتی (بخش ۹)، مانند جداسازی داده‌ی کاربران و الزام گذرواژه‌ی فعلی، تست‌های اختصاصی نوشته شده است.

۸. CI (یکپارچه‌سازی مداوم)

فایل `.github/workflows/ci.yml` در هر push به Branch اصلی و در هر Pull Request، مراحل زیر را روی Ubuntu و Node.js 24 اجرا می‌کند:

دریافت کد ◄ npm ci ◄ Lint ◄ Build ◄ Test + Coverage ◄ ذخیره‌ی گزارش Coverage

در صورت شکست هر مرحله، Workflow متوقف و مشکل پیش از ادغام کد آشکار می‌شود. گزارش Coverage به‌مدت ۷ روز به‌عنوان Artifact در Workflow نگه‌داری می‌شود.

۹. ملاحظات امنیتی

۱ جداسازی داده‌ی کاربران

همه‌ی متدهای سرویس‌ها به شناسه‌ی مالک محدود هستند؛ بنابراین هیچ کاربری با حدس شناسه نمی‌تواند به وظایف، دسته‌بندی‌ها یا اعلان‌های کاربر دیگر دسترسی پیدا کند. دسترسی به داده‌ی دیگران خطای **۴۰۴** (نه ۴۰۳) برمی‌گرداند تا وجود داده افشا نشود.

۲ تغییر امن گذرواژه

ویرایش پروفایل فقط از طریق روت `PATCH /users/me` انجام می‌شود و هویت کاربر از توکن خوانده می‌شود (نه از آدرس). برای تغییر گذرواژه، ارسال گذرواژه‌ی فعلی الزامی است و هم در DTO و هم در سرویس بررسی می‌شود.

سایر تدابیر امنیتی:

- ذخیره‌ی گذرواژه‌ها به‌صورت هش‌شده با bcrypt و عدم نمایش آن‌ها در هیچ پاسخی.
- پیام خطای یکسان برای ایمیل ناموجود و گذرواژه‌ی نادرست در ورود.
- بازخوانی کاربر در هر درخواست تا توکن حساب حذف‌شده فوراً بی‌اعتبار شود.
- رد Propertyهای ناشناخته در Body درخواست (`forbidNonWhitelisted`).
- محدودسازی فیلد مرتب‌سازی به فهرست مجاز و سقف ۱۰۰ برای اندازه‌ی صفحه.
- خواندن Secret از متغیر محیطی، اعتبارسنجی آن در زمان راه‌اندازی و خارج‌بودن `.env` از ریپازیتوری.